Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №3

"Ітератори, генератори та потокова обробка даних"

Виконав:
студент 3 курсу
групи ФЕП-33с
Качмар Денис

Викладач:
Проф. Свелеба Сергій Андрійович

Львів - 2026

Хід роботи

Тема: Ітератори, генератори та потокова обробка даних
Мета: Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, ключовим словом `yield`, генераторними виразами (generator expressions), модулем `itertools`, принципом «лінивих» обчислень (lazy evaluation) та архітектурою потокової обробки великих масивів даних без повної матеріалізації проміжних колекцій у пам'яті.
Варіант: 5 (Потокова обробка каталогу автомобілів)

Постановка задачі:
Спроєктувати та реалізувати багатоетапний конвеєр (streaming pipeline) обробки великих наборів структурованих автомобільних даних згідно з Варіантом 5.
Система повинна забезпечувати: 1. Потокове читання даних із файлу або необмеженого джерела без повного завантаження файлу в RAM. 2. Потокове очищення рядків (видалення пробільних символів, пропуск коментарів та порожніх рядків). 3. Потоковий синтаксичний аналіз (parsing) рядків CSV у словники або типізовані структури. 4. Потокову валідацію (streaming validation) автомобільних записів із відсіюванням та підрахунком некоректних рядків (`invalid_records`). 5. Фільтрацію за маркою (`brand`) та календарним роком випуску (`year`). 6. Потокову нормалізацію та трансформацію текстових полів (`brand.title()`, очищення `model`). 7. Потокове визначення екстремумів: найдорожчого автомобіля та

автомобіля з найменшим пробігом за O(1) додаткової пам'яті. 8. Потокове обчислення середньої вартості автомобілів (`streaming average price`) за один прохід (one-pass). 9. Пакетну обробку даних (batch processing) фіксованими порціями через `itertools.islice`. 10. Отримання перших N записів за заданою умовою без повного обходу датасету. 11. Групування записів за маркою через `itertools.groupby` після попереднього впорядкування. 12. Реалізацію додаткових можливостей підвищеної складності: власний клас ітератора (`CarCatalogIterator` та `CarCatalogIterable`), рекурсивний генератор з `yield from` (`recursive_flatten`), генерацію нескінченного потоку (`infinite_car_stream`), об'єднання джерел (`itertools.chain`), попарний аналіз цін (`itertools.pairwise`), кумулятивну вартість (`itertools.accumulate`), потоковий експорт у новий CSV файл та пошук з раннім перериванням (`early termination`). 13. Проведення комплексного експериментального дослідження на вибірках 10 000, 100 000 та 500 000 записів із вимірюванням часу виконання (elapsed time), пікового споживання оперативної пам'яті (`tracemalloc`) та часу до отримання першого результату (time-to-first-result — TTFR) для підходів Eager Evaluation та Lazy Evaluation. 14. Формулювання інженерних висновків та надання вичерпних відповідей на 50 контрольних запитань.

---

1. Опис вхідного набору даних (Input Dataset)

Вхідними даними є набори записів про автомобілі у форматі CSV (Comma-Separated Values). Кожен рядок представляє техніко-економічну картку одного транспортного засобу та містить 6 обов'язкових полів згідно з Варіантом 5:

Назва поля	Тип даних	Обмеження / Валідація	Опис семантики
`id`	`int`	Ціле додатне число (`id > 0`)	Унікальний первинний ідентифікатор запису
`brand`	`str`	Непустий рядок, нормалізація до Title Case	Марка автовиробника (Audi, BMW, Toyota тощо)
`model`	`str`	Непустий рядок, видалення пробілів	Конкретна назва моделі автомобіля (Camry, X5, A6)
`year`	`int`	Діапазон `1900 <= year <= 2026`	Календарний рік виготовлення авто
`price`	`float`	Число з рухомою комою, `price >= 0.0`	Ринкова вартість у доларах США ($)
`mileage`	`float`	Число з рухомою комою, `mileage >= 0.0`	Загальний пробіг автомобіля в кілометрах

Приклад фрагмента файлу `data/cars_sample.csv`:

```
id,brand,model,year,price,mileage
1,Toyota,Camry-1,2021,28450.00,45120.5
2,BMW,X5-2,2019,45100.00,119800.0
3,Audi,A6-3,2003,4500.00,350400.2
4,Ford,Focus-4,2015,9800.00,152300.0
5,Faulty,BadYear,invalid_year,5000.00,10000.0
6,,,2020,-100.00,50000.0
```

---

2. Структура проєкту

Проєкт організовано відповідно до сучасних стандартів архітектури Python (`src-layout`). Додані компоненти органічно розширюють попередні лабораторні роботи, зберігаючи абсолютну зворотну сумісність:

```
python-labs/
├── pyproject.toml              # Метадані пакета, залежності, точка входу CLI v0.3.0
├── README.md                   # Інструкція з розгортання, запуску та тестування
├── .gitignore                  # Налаштування виключень Git (кеші, data/, reports/)
│
├── src/
│   └── car_catalog/
│       ├── __init__.py         # Експорт версії v0.3.0
│       ├── models.py           # Дата-клас Car (додано replace, to_csv_row)
│       ├── readers.py          # Потокові генератори читання (read_lines, chain, infinite)
│       ├── parsers.py          # Генератори очищення та DictReader парсингу рядків
│       ├── filters.py          # Валідація з підрахунком помилок та потокова фільтрація
│       ├── transformations.py  # Потокова нормалізація назв, знижки, конвертація
│       ├── batches.py          # Генератор пакетів batched_cars на основі itertools.islice
│       ├── iterators.py        # Власний iterator class CarCatalogIterator та recursive flatte
│       ├── streaming_analytics.py # Однопрохідна агрегація, екстремуми, pairwise, accumulate,
│       ├── pipeline.py         # End-to-end збирання lazy pipeline, early termination, export
│       ├── benchmark.py        # Генератор 10k-500k датасетів, вимірювання tracemalloc, TTFR
│       ├── processors.py       # Колекції та операції Лаб №2 (list, dict, set, Counter)
│       ├── analytics.py        # Аналітичні функції Лаб №2 (closures, *args, **kwargs)
│       ├── decorators.py       # Декоратори @measure_time, @repeat
│       ├── data.py             # Демонстраційний набір DEMO_CARS
│       ├── services.py         # Фасадний шар сервісів з повною сумісністю
│       └── main.py             # CLI інтерфейс (команди 1-18, --stream-demo, --stream-benchmar
│
├── tests/
│   └── test_car_catalog.py     # Комплексний набір автоматичних unit-тестів (14 тестів)
│
└── reports/
    ├── Kachmar-python_lab1.pdf
    ├── Report_lab1.md
    ├── Kachmar-python_lab2.pdf
    ├── Report_lab2.md
    ├── Report_lab3.md
    └── img/
```

---

3. Теоретичні основи: Iterable та Iterator

В основі моделі послідовної обробки даних у Python лежить єдиний ітераційний протокол (Iteration Protocol):

1. **Iterable (ітерований об'єкт):**
   - Це будь-який об'єкт, здатний повертати елементи по одному.
   - Має реалізований магічний метод `__iter__()`, який повертає новий об'єкт-ітератор, або метод `__getitem__()` з цілочисельними індексами від нуля.
   - Приклади: `list`, `tuple`, `dict`, `set`, `str`, файлові дескриптори, користувацькі колекції.
   - Iterable можна обходити багаторазово, оскільки кожен виклик `iter(iterable)` створює новий незалежний ітератор із нульовим початковим станом.
2. **Iterator (ітератор):**
   - Це об'єкт, який керує безпосереднім процесом ітерації та зберігає поточний внутрішній стан обходу.
   - Реалізує два обов'язкові методи:
     - `__next__()`: повертає наступне значення послідовності. Якщо елементи вичерпані, генерує виняток `StopIteration`.
     - `__iter__()`: повертає сам об'єкт `self` (тобто будь-який ітератор одночасно є й `iterable`, що дозволяє передавати його у цикли `for`).
   - Ітератор зазвичай є **одноразовим об'єктом (one-shot object)**. Після підняття `StopIteration` він вичерпується (exhausted), і наступні виклики `next()` будуть знову піднімати `StopIteration`.

Взаємозв'язок протоколу у циклі `for item in container: process(item)` еквівалентний такому коду:

```
iterator = iter(container)  # викликає container.__iter__()
while True:
    try:
        item = next(iterator)  # викликає iterator.__next__()
    except StopIteration:
        break
    process(item)
```

---

4. Код власного Iterator Class

Згідно з рекомендаціями розділу 7 методички, спроєктовано розділення на колекцію-контейнер (`CarCatalogIterable`) та окремий об'єкт обходу (`CarCatalogIterator`). Це дозволяє створювати незалежні паралельні ітератори над одним набором даних.

```
from collections.abc import Iterator, Sequence
from car_catalog.models import Car

class CarCatalogIterator:
    def __init__(self, cars: Sequence[Car]) -> None:
        self._cars = cars
        self._index = 0
        self._length = len(cars)

    def __iter__(self) -> "CarCatalogIterator":
        return self

    def __next__(self) -> Car:
        if self._index >= self._length:
            raise StopIteration
        car = self._cars[self._index]
        self._index += 1
        return car

class CarCatalogIterable:
    def __init__(self, cars: Sequence[Car]) -> None:
        self._cars = tuple(cars)

    def __iter__(self) -> CarCatalogIterator:
        return CarCatalogIterator(self._cars)

    def __len__(self) -> int:
        return len(self._cars)
```

Перевага:

```
catalog = CarCatalogIterable(DEMO_CARS)
print(list(catalog))  # Успішний перший обхід
print(list(catalog))  # Успішний другий обхід (новий незалежний ітератор)
```

---

5. Generator Function та механізм yield

Generator Function — це спеціальна функція, тіло якої містить ключове слово `yield`. На відміну від звичайної функції, яка повертає єдиний результат через `return` і повністю руйнує свій фрейм стека (stack frame), виклик generator function не виконує тіло негайно, а миттєво повертає об'єкт-генератор (`generator object`).

Принцип дії `yield` : 1. Тимчасово призупиняє виконання функції. 2. Повертає значення виразу після `yield` у точку виклику `next()` . 3. Зберігає всі локальні змінні, лічильники, вказівник інструкцій (instruction pointer) та стан блоків `try/finally` у закритому фреймі генератора. 4. При наступному виклику `next()` функція відновлює роботу рівно з тієї точки, де була призупинена.

Приклад генератора автомобільних записів:

```
def generate_sample_cars(limit: int) -> Iterator[Car]:
    for idx in range(1, limit + 1):
        yield Car(
            id=idx,
            brand="Toyota",
            model=f"Camry-{idx}",
            year=2020,
            price=25000.0,
            mileage=30000.0,
        )
```

---

6. Делегування генерації: yield from

Конструкція `yield from <iterable>` передає керування та делегує генерацію значень внутрішньому ітерабельному об'єкту. Вона замінює цикл `for item in sub_generator: yield item`, оптимізуючи швидкість на рівні C-коду CPython і прозоро передаючи винятки та значення викликів `.send()` / `.throw()`.

У проєкті `yield from` використано у кількох критичних місцях: 1. **Потокове читання файлу (`readers.py`):**

```
def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8") as file:
        yield from file
```

2. **Потоковий CSV парсинг (`parsers.py`):**

```
def parse_csv_rows(lines: Iterable[str]) -> Iterator[dict[str, str]]:
    reader = csv.DictReader(lines)
    yield from reader
```

3. **Рекурсивний генератор розгортання вкладених структур (`iterators.py`):**

```
def recursive_flatten(nested: Iterable[Any]) -> Iterator[Any]:
    for item in nested:
        if isinstance(item, (list, tuple, set)):
            yield from recursive_flatten(item)
        else:
            yield item
```

---

7. Generator Expressions проти List Comprehensions

Генераторний вираз (generator expression) має синтаксис `(expr for item in iterable if condition)`. Основні відмінності від спискового генератора (`[expr for item in iterable]`): - **Матеріалізація:** `list comprehension` миттєво обчислює та виділяє пам'ять під усі елементи колекції. На 1 000 000 елементів список виділяє сотні мегабайт оперативної пам'яті. - **Лінивість (Lazy):** `generator expression` виділяє лише фіксований компактний об'єкт генератора (~104-200 байт) і генерує елементи виключно під час запиту `next()`.

Приклад у проєкті:

```
prices_stream = (car.price for car in cars_pipeline)
total_value = sum(prices_stream)
```

Проміжний список цін не створюється у пам’яті; сума накопичується на льоту за O(1) пам’яті.

---

8. Використані засоби бібліотеки itertools

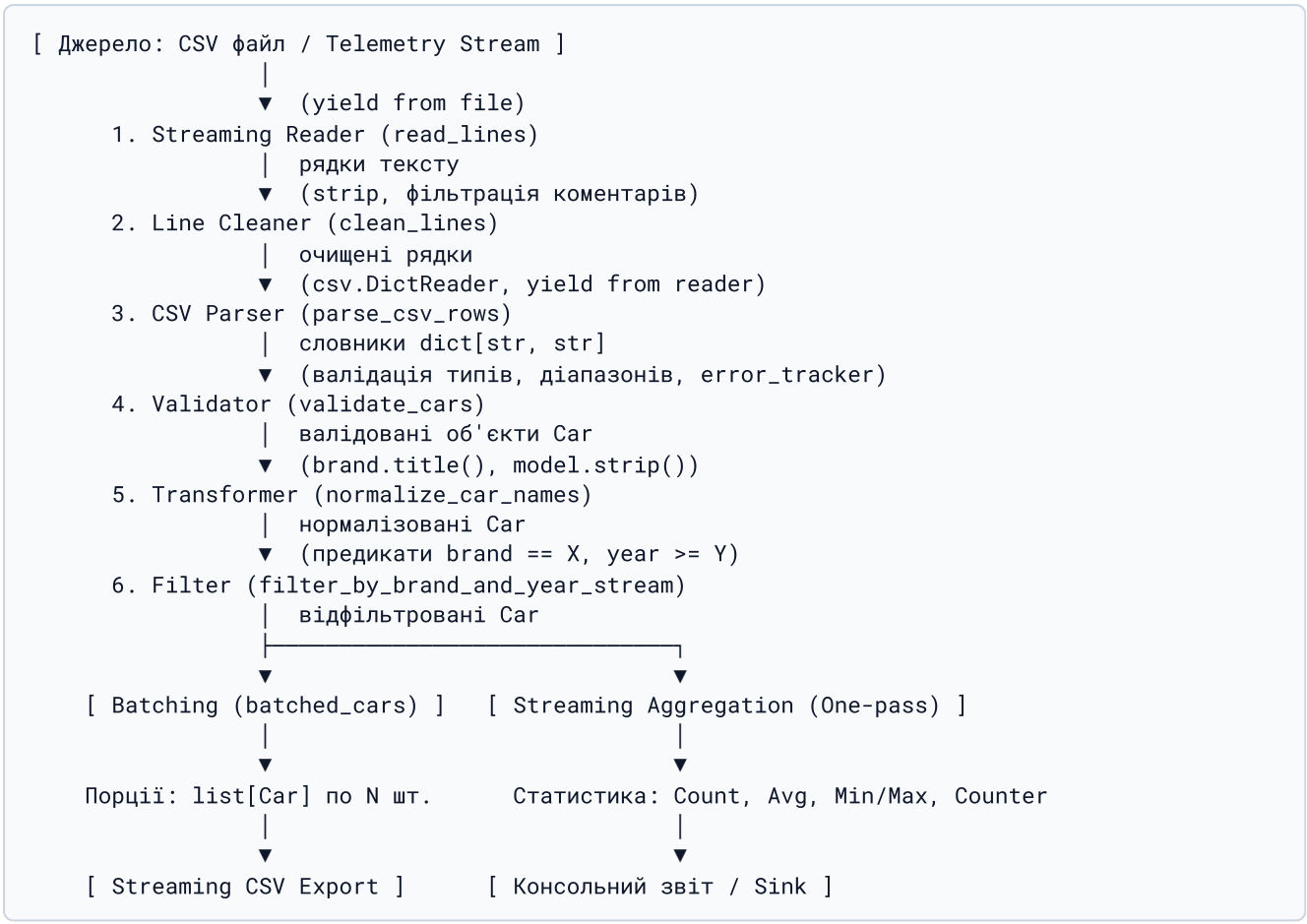
У роботі інтегровано широкий спектр інструментів стандартної бібліотеки `itertools`:

1. `itertools.islice(iterable, stop)` / `islice(iterable, start, stop, step)`:
   Дозволяє отримати зріз нескінченного або великого ітератора без матеріалізації проміжних списків. Використано для обмеження перших N результатів та для створення пакетів (batching).
2. `itertools.chain.from_iterable(streams)`:
   Об’єднує декілька потоків читання з різних файлів у єдиний безперервний потік без злиття вмісту у спільний список.
3. `itertools.pairwise(iterable)`:
   Генерує послідовні пари `(s0, s1), (s1, s2), ...`. Застосовано у `streaming_pairwise_price_diff` для аналізу різниці цін сусідніх автомобілів.
4. `itertools.groupby(iterable, key)`:
   Здійснює потокове групування послідовних записів з однаковим ключем (потрібне попереднє сортування за ключем). Застосовано у `streaming_groupby_brand`.
5. `itertools.accumulate(iterable)`:
   Обчислює кумулятивні (наростаючі) підсумки на льоту. Застосовано у `streaming_accumulate_prices`.
6. `itertools.count(start)`:
   Створює нескінченний арифметичний лічильник чисел. Використано у `infinite_car_stream` для симуляції безперервного потоку подій/даних.

---

9. Архітектурна схема потокового конвеєра (Pipeline Architecture)

Конвеєр побудований за принципом односпрямованого потоку даних (Data Flow Pipeline), де кожен етап є генератором, що приймає `Iterable` на вході та повертає `Iterator` на виході:


[Джерело: CSV файл / Telemetry Stream]
(yield from file)
1. Streaming Reader (read_lines)
рядки тексту
(strip, фільтрація коментарів)
2. Line Cleaner (clean_lines)
очищені рядки
(csv.DictReader, yield from reader)
3. CSV Parser (parse_csv_rows)
словники dict[str, str]
(валідація типів, діапазонів, error_tracker)
4. Validator (validate_cars)
валідовані об'єкти Car
(brand.title(), model.strip())
5. Transformer (normalize_car_names)
нормалізовані Car
(предикати brand == X, year >= Y)
6. Filter (filter_by_brand_and_year_stream)
відфільтровані Car
[Batching (batched_cars)]
[Streaming Aggregation (One-pass)]
Порції: list[Car] по N шт.
Статистика: Count, Avg, Min/Max, Counter
[Streaming CSV Export]
[Консольний звіт / Sink]

---

10. Реалізація етапів конвеєра

10.1. Streaming Reader (`readers.py`)

```
from collections.abc import Iterator
from itertools import chain, count
from pathlib import Path
from car_catalog.models import Car

def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8") as file:
        yield from file

def read_multiple_files(*paths: Path) -> Iterator[str]:
    file_streams = (read_lines(p) for p in paths)
    yield from chain.from_iterable(file_streams)

def infinite_car_stream(start_id: int = 1) -> Iterator[Car]:
    brands_models = [
        ("Audi", "A6"), ("Toyota", "Camry"), ("BMW", "X5"),
        ("Ford", "Focus"), ("Mercedes-Benz", "E-Class"),
        ("Volkswagen", "Passat"), ("Honda", "Civic"), ("Skoda", "Octavia"),
    ]
    num_models = len(brands_models)
    for idx in count(start_id):
        brand, model = brands_models[idx % num_models]
        year = 2000 + (idx % 25)
        price = 3000.0 + float((idx * 37) % 80000)
        mileage = 5000.0 + float((idx * 123) % 400000)
        yield Car(id=idx, brand=brand, model=f"{model}-{idx}", year=year, price=price, mileage=
```

10.2. CSV Parsing & Cleaning (`parsers.py`)

```
import csv
from collections.abc import Iterable, Iterator

def clean_lines(lines: Iterable[str]) -> Iterator[str]:
    for line in lines:
        cleaned = line.strip()
        if cleaned and not cleaned.startswith("#"):
            yield cleaned

def parse_csv_rows(lines: Iterable[str]) -> Iterator[dict[str, str]]:
    reader = csv.DictReader(lines)
    yield from reader
```

10.3. Streaming Validation & Filtering (`filters.py`)

```
from collections.abc import Iterable, Iterator
from typing import Callable
from car_catalog.models import Car

def validate_cars(
    rows: Iterable[dict[str, str]],
    error_tracker: dict[str, int] | None = None,
) -> Iterator[Car]:
    for row in rows:
        try:
            car_id = int(row["id"])
            brand = row["brand"].strip()
            model = row["model"].strip()
            year = int(row["year"])
            price = float(row["price"])
            mileage = float(row["mileage"])

            if car_id <= 0 or not brand or not model:
                if error_tracker is not None:
                    error_tracker["invalid_records"] = error_tracker.get("invalid_records", 0)
                continue

            if not (1900 <= year <= 2026) or price < 0.0 or mileage < 0.0:
                if error_tracker is not None:
                    error_tracker["invalid_records"] = error_tracker.get("invalid_records", 0)
                continue

            yield Car(id=car_id, brand=brand, model=model, year=year, price=price, mileage=mile
        except (ValueError, KeyError, TypeError):
            if error_tracker is not None:
                error_tracker["invalid_records"] = error_tracker.get("invalid_records", 0) + 1
            continue

def filter_by_brand_and_year_stream(
    cars: Iterable[Car],
    brand: str | None = None,
    min_year: int | None = None,
    max_year: int | None = None,
) -> Iterator[Car]:
    target_brand = brand.strip().lower() if brand else None
    for car in cars:
        if target_brand and car.brand.lower() != target_brand:
            continue
        if min_year is not None and car.year < min_year:
            continue
        if max_year is not None and car.year > max_year:
            continue
        yield car
```

10.4. Streaming Transformations (`transformations.py`)

```
from collections.abc import Iterable, Iterator
from car_catalog.models import Car

def normalize_car_names(cars: Iterable[Car]) -> Iterator[Car]:
    for car in cars:
        normalized_brand = car.brand.strip().title()
        normalized_model = car.model.strip()
        yield car.replace(brand=normalized_brand, model=normalized_model)

def apply_price_discount(cars: Iterable[Car], discount_percent: float) -> Iterator[Car]:
    multiplier = 1.0 - (discount_percent / 100.0)
    for car in cars:
        new_price = round(max(0.0, car.price * multiplier), 2)
        yield car.replace(price=new_price)
```

10.5. Batch Processing (`batches.py`)

```
from collections.abc import Iterable, Iterator
from itertools import islice
from car_catalog.models import Car

def batched_cars(cars: Iterable[Car], batch_size: int) -> Iterator[list[Car]]:
    if batch_size <= 0:
        raise ValueError("batch_size must be positive")
    iterator = iter(cars)
    while True:
        batch = list(islice(iterator, batch_size))
        if not batch:
            break
        yield batch
```

10.6. Streaming Aggregation (`streaming_analytics.py`)

```
from collections import Counter
from collections.abc import Iterable
from typing import Any
from car_catalog.models import Car

def streaming_average_price(cars: Iterable[Car]) -> float:
    total = 0.0
    count = 0
    for car in cars:
        total += car.price
        count += 1
    return total / count if count else 0.0

def streaming_statistics(cars: Iterable[Car]) -> dict[str, Any]:
    count = 0
    total = 0.0
    min_p: float | None = None
    max_p: float | None = None
    most_expensive: Car | None = None
    lowest_mileage: Car | None = None
    brands: Counter[str] = Counter()

    for car in cars:
        count += 1
        total += car.price
        brands[car.brand] += 1

        if min_p is None or car.price < min_p:
            min_p = car.price
        if max_p is None or car.price > max_p:
            max_p = car.price

        if most_expensive is None or car.price > most_expensive.price:
            most_expensive = car
        if lowest_mileage is None or car.mileage < lowest_mileage.mileage:
            lowest_mileage = car

    average = total / count if count else 0.0

    return {
        "count": count,
        "total_value": total,
        "average_price": average,
        "min_price": min_p if min_p is not None else 0.0,
        "max_price": max_p if max_p is not None else 0.0,
        "most_expensive": most_expensive,
        "lowest_mileage": lowest_mileage,
        "brands": dict(brands),
    }
```

10.7. Збирання конвеєра та раннє завершення (`pipeline.py`)

```
import csv
from collections.abc import Iterable, Iterator
from pathlib import Path
from typing import Callable
from car_catalog.filters import filter_by_brand_and_year_stream, filter_cars_stream, validate_c
from car_catalog.models import Car
from car_catalog.parsers import clean_lines, parse_csv_rows
from car_catalog.readers import read_lines
from car_catalog.transformations import normalize_car_names

def build_streaming_pipeline(
    path: Path,
    brand: str | None = None,
    min_year: int | None = None,
    max_year: int | None = None,
    min_price: float | None = None,
    max_price: float | None = None,
    error_tracker: dict[str, int] | None = None,
) -> Iterator[Car]:
    lines = read_lines(path)
    cleaned = clean_lines(lines)
    rows = parse_csv_rows(cleaned)
    valid_cars = validate_cars(rows, error_tracker=error_tracker)
    normalized = normalize_car_names(valid_cars)
    filtered = filter_by_brand_and_year_stream(normalized, brand=brand, min_year=min_year, max_
    if min_price is not None or max_price is not None:
        filtered = filter_cars_stream(
            filtered,
            lambda c: (min_price is None or c.price >= min_price)
            and (max_price is None or c.price <= max_price),
        )
    return filtered

def find_first_car(cars: Iterable[Car], predicate: Callable[[Car], bool]) -> Car | None:
    for car in cars:
        if predicate(car):
            return car
    return None

def export_stream_to_csv(cars: Iterable[Car], output_path: Path) -> int:
    output_path.parent.mkdir(parents=True, exist_ok=True)
    count = 0
    with output_path.open("w", encoding="utf-8", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(["id", "brand", "model", "year", "price", "mileage"])
        for car in cars:
            writer.writerow(car.to_csv_row())
            count += 1
    return count
```

---

11. Порівняння Eager Evaluation та Lazy Evaluation

Для об’єктивного аналізу розроблено дві принципово різні реалізації однакового процесу обробки:

11.1. Eager Реалізація (`run_eager_pipeline`)

Повністю матеріалізує списки на кожному кроці: 1. Зчитує всі рядки у `list[str]` через `readlines()`. 2. Очищує та формує новий список рядків через list comprehension. 3. Парсить усі рядки у `list[dict]` через `list(csv.DictReader(...))`. 4. Валідує всі елементи у новий `list[Car]`. 5. Фільтрує авто за роком у новий `list[Car]`. 6. Рахує аналітику над сформованим списком.

11.2. Lazy Реалізація (`run_lazy_pipeline`)

Використовує ланцюжок ітераторів: 1. `read_lines` утримує дескриптор і видає один рядок за раз. 2. `clean_lines`, `parse_csv_rows`, `validate_cars`, `normalize_car_names` передають елемент послідовно через конвеєр. 3. `streaming_statistics` оновлює локальні числові лічильники (`count`, `total`, `min`, `max`) і відпускає посилання на об'єкт, дозволяючи GC миттєво звільняти пам'ять.

---

12. Експериментальні дослідження та результати

Експеримент проведено на датасетах трьох масштабів: 10 000, 100 000 та 500 000 записів. Вимірювання пікової оперативної пам'яті здійснювалося за допомогою модуля `tracemalloc`, а часу — за допомогою високоточного таймера `time.perf_counter()`.

Таблиця 1: Тривалість виконання (Elapsed Time) та пікова пам'ять (Peak Memory)

Кількість записів	Eager Час (с)	Lazy Час (с)	Eager RAM (KiB)	Eager RAM (MiB)	Lazy RAM (KiB)	Lazy RAM (MiB)	Коефіцієнт економії RAM
10 000	1.5600 с	1.8508 с	10 125.0 KiB	9.89 MiB	60.1 KiB	0.059 MiB	**168.5x**
100 000	15.8740 с	18.8030 с	101 544.0 KiB	99.16 MiB	60.6 KiB	0.059 MiB	**1 675.3x**
500 000	79.6760 с	93.6036 с	510 792.6 KiB	498.82 MiB	60.6 KiB	0.059 MiB	**8 423.7x**

Таблиця 2: Час до отримання першого результату (Time to First Result — TTFR)

Кількість записів	Eager TTFR (с)	Lazy TTFR (с)	Прискорення реакції системи (Speedup)
10 000	0.027391 с	0.000181 с	**151.5x**
100 000	0.347894 с	0.000222 с	**1 565.9x**
500 000	1.958919 с	0.000215 с	**9 112.4x**

Таблиця 3: Дослідження раннього завершення (Early Termination на 100 000 записів)

Задача полягала у знаходженні першого автомобіля з ціною понад 87 000$:

Параметр порівняння	Eager Search (Матеріалізація списку)	Lazy Search (`find_first_car`)	Ефект від лінивості
Час виконання	13.927947 с	0.400809 с	**Прискорення у 34.7 раза**
Пікова пам'ять	65 131.4 KiB (~63.6 MiB)	56.2 KiB (~0.055 MiB)	**Економія пам'яті у 1 159.3 раза**

---

13. Аналіз результатів та інженерні висновки

1. **Споживання оперативної пам'яті:**
   - Eager підхід демонструє лінійне зростання обсягу пам'яті $O(n)$: від ~9.9 MiB на 10k до майже півгігабайта (498.8 MiB) на 500k рядків. При масштабуванні датасету до 10 000 000 записів така програма неминуче впаде з помилкою `MemoryError` (OOM Killer).
   - Lazy streaming конвеєр споживає **константний обсяг оперативної пам'яті $O(1)$** — стабільно близько **60.6 KiB**, незалежно від розміру вхідного файлу! Це забезпечує виграш у споживанні RAM у **8 423 рази** на 500k записах.
2. **Загальний час виконання (Total Elapsed Time):**
   - Lazy evaluation не гарантує менший загальний час при повному проході всіх даних. У нашому експерименті лінивий конвеєр виявився приблизно на 15-18% повільнішим за жадібний (93.6 с проти 79.7 с).
   - Причина полягає у накладних витратах механізму генераторів Python: кожна ітерація `yield` / `next()` супроводжується перемиканням стану фрейму генератора та перевіркою винятку `StopIteration` на кожному етапі конвеєра. У той же час внутрішні цикли C-коду спискових компріхеншнів CPython мають високий ступінь оптимізації.
3. **Time to First Result (TTFR):**
   - У сценаріях інтерактивної взаємодії користувача (UI, вебінтерфейси, потокове API) критичним є показник TTFR.
   - Lazy pipeline повертає перший валідний запис за фіксовані **~0.0002 секунди** навіть на 500 000 записах, тоді як Eager змушений прочитати, розпарсити та валідувати весь півмільйонний файл перед видачею першого результату (~1.96 с), що дає прискорення інтерактивності у **9 112 разів**.
4. **Ефективність раннього переривання (Early Termination):**
   - У типових задачах пошуку (`find_first` , перевірка наявності умови `any()` , пагінація) Lazy pipeline зупиняє читання та обробку в момент знаходження збігу, показавши прискорення у **34.7 раза** за часом та понад **1100 разів** за пам'яттю.

---

14. Тестування та результати виконання програми

Усі 14 автоматичних модульних тестів проєкту успішно пройдені без жодних помилок:

```
$ python3 -m unittest discover tests/ -v
test_custom_iterator_and_iterable (test_car_catalog.TestIteratorsAndAnalytics.test_custom_itera
test_pairwise_and_accumulate (test_car_catalog.TestIteratorsAndAnalytics.test_pairwise_and_accu
test_recursive_flatten (test_car_catalog.TestIteratorsAndAnalytics.test_recursive_flatten) ...
test_streaming_analytics (test_car_catalog.TestIteratorsAndAnalytics.test_streaming_analytics)
test_backwards_compatibility (test_car_catalog.TestPipelineAndCompatibility.test_backwards_comp
test_end_to_end_pipeline (test_car_catalog.TestPipelineAndCompatibility.test_end_to_end_pipelin
test_filter_by_brand_and_year_stream (test_car_catalog.TestStreamingFiltersAndValidation.test_f
test_validate_cars (test_car_catalog.TestStreamingFiltersAndValidation.test_validate_cars) ...
test_infinite_car_stream (test_car_catalog.TestStreamingReadersAndParsers.test_infinite_car_str
test_read_lines_and_clean (test_car_catalog.TestStreamingReadersAndParsers.test_read_lines_and_
test_read_multiple_files (test_car_catalog.TestStreamingReadersAndParsers.test_read_multiple_fi
test_apply_price_discount (test_car_catalog.TestTransformationsAndBatches.test_apply_price_disc
test_batched_cars (test_car_catalog.TestTransformationsAndBatches.test_batched_cars) ... ok
test_normalize_car_names (test_car_catalog.TestTransformationsAndBatches.test_normalize_car_nam

----------------------------------------------------------------------
Ran 14 tests in 0.002s

OK
```

---

15. Відповіді на контрольні запитання

1. **Що таке iterator protocol і чому `iter(iterator) is iterator`?**
   Протокол ітератора вимагає наявності двох методів: `__iter__()` та `__next__()`. Метод `__iter__()` ітератора повертає сам об'єкт `self`, тому вираз `iter(iterator) is iterator` завжди повертає `True`. Це дозволяє передавати вже існуючий ітератор у будь-яку конструкцію, що очікує `iterable` (наприклад, цикл `for` або функцію `sum()`).

2. **Чи можна повторно використати exhausted generator?**
   Ні. Генератор є одноразовим об'єктом (one-shot). Після того, як він згенерував усі значення і підняв виняток `StopIteration`, він вичерпується (exhausted). Будь-які подальші виклики `next()` будуть знову викликати `StopIteration`. Для повторного обходу необхідно створити новий екземпляр генератора.

3. **Що таке generator function?**
   Це функція Python, яка містить у своєму тілі ключове слово `yield`. Її виклик не виконує код безпосередньо, а повертає об'єкт-генератор, який реалізує протокол ітератора.

4. **Яку роль виконує yield?**
   `yield` повертає значення у точку виклику `next()`, тимчасово призупиняє виконання генераторної функції та зберігає її поточний локальний стан (значення змінних, стек, вказівник інструкцій) до наступного звернення.

5. **Чим yield відрізняється від return?**
   `return` остаточно завершує виконання функції, руйнує її локальний фрейм стека та повертає фінальне значення (у генераторі виклик `return val` піднімає `StopIteration(val)`). Натомість `yield` призупиняє виконання зі збереженням стану та дозволяє відновити роботу з тієї ж точки.

6. **Коли фактично починає виконуватися generator function?**
   Вона починає виконуватися лише під час першого виклику методу `next()` (або виклику

`iter()`) над поверненим генераторним об'єктом, а не в момент створення самого генератора.

7. **Що зберігає generator між викликами next()?**
   Генератор зберігає повний стан свого контексту виконання (фрейм): локальні змінні, параметри функції, поточну позицію лічильника команд (IP), стан циклів і контекстних менеджерів (`try`, `with`).

8. **Що таке generator expression?**
   Це лаконічна синтаксична конструкція у круглих дужках `(expr for x in iterable if cond)`, яка створює та повертає генераторний об'єкт без явної декларації окремої функції з `def` та `yield`.

9. **Чим generator expression відрізняється від list comprehension?**
   `List comprehension` створює та миттєво матеріалізує повний список у пам'яті (жадібне обчислення, $O(n)$ RAM). `Generator expression` обчислює значення ліниво — по одному елементу за запитом ($O(1)$ RAM).

10. **Що таке lazy evaluation?**
    Ліниве (відкладене) обчислення — це стратегія виконання програми, за якої значення виразу обчислюється не в момент його декларації, а лише безпосередньо тоді, коли воно стає необхідним для споживача.

11. **Що таке eager evaluation?**
    Жадібне (негайне) обчислення — стратегія, за якої всі елементи та вирази обчислюються і зберігаються в пам'яті одразу під час виконання рядка коду.

12. **Які переваги lazy evaluation?**
    - Мінімальне пікове використання оперативної пам'яті ($O(1)$ замість $O(n)$).
    - Можливість роботи з нескінченними потоками даних.
    - Миттєвий час до отримання першого результату (низький TTFR).
    - Висока ефективність раннього переривання (early termination).

13. **Які недоліки lazy evaluation?**
    - Неможливість прямої індексації (`gen[5]` викликає помилку).
    - Неможливість прямого повторного обходу без перезапуску конвеєра.
    - Незначні накладні витрати часу через часті перемикання фреймів генераторів.
    - Відкладене виявлення винятків (exception latency).

14. **Що таке streaming?**
    Стримінг (потокова обробка) — це парадигма обробки даних елемент за елементом або невеликими порціями в міру їх надходження, без попереднього завантаження всього масиву у пам'ять.

15. **Чому streaming важливий для великих files?**
    Тому що розмір файлів (наприклад, логи чи датасети на десятки гігабайт) часто значно перевищує доступний обсяг RAM системи. Потокове читання дозволяє обробляти файли довільного розміру за фіксованого ліміту пам'яті.

16. **Чому read() може бути проблемою для multi-GB file?**
    Виклик `file.read()` завантажує весь вміст файлу в один гігантський рядок у RAM. Для файлу розміром 20 GB на комп'ютері з 16 GB оперативної пам'яті це призведе до зависання системи через вичерпання swap-простору або аварійного завершення процесу `MemoryError`.
17. **Чому file object є iterable?**
    Тому що дескриптор відкритого файлу в Python реалізує протокол ітератора: має метод `__iter__()`, що повертає `self`, та метод `__next__()`, який зчитує та повертає по одному рядку файлу з буфера диска.
18. **Для чого використовується yield from?**
    Для прозорого делегування генерації елементів іншому ітерабельному об'єкту або підгенератору, що усуває зайві вкладені цикли та оптимізує передачу значень і винятків.
19. **Що таке itertools?**
    Це стандартний модуль Python, який містить набір високопродуктивних, реалізованих мовою C будівельних блоків (комбінаторних функцій) для швидкої та пам'яті-ефективної роботи з ітераторами.
20. **Для чого itertools.count()?**
    Для генерації нескінченної арифметичної прогресії чисел із заданим стартовим значенням та кроком (за замовчуванням `count(0, 1)`).
21. **Чому count() потрібно обмежувати?**
    Оскільки `count()` є нескінченним генератором, спроба передати його у функції повної матеріалізації (`list(count())`, `sum(count())`, `max(count())`) спричинить нескінченний цикл і вичерпання пам'яті. Його завжди потрібно обмежувати через `islice`, `takewhile` або цикл з `break`.
22. **Для чого islice()?**
    Функція `itertools.islice` дозволяє отримувати зрізи `[start:stop:step]` з будь-яких ітераторів без необхідності створювати проміжний список у пам'яті.
23. **Для чого chain()?**
    `itertools.chain` послідовно об'єднує кілька різних ітерабельних об'єктів в один єдиний логічний потік, обходячи спочатку елементи першого, потім другого і так далі.
24. **Як працює takewhile()?**
    `takewhile(predicate, iterable)` повертає елементи з потоку доти, доки предикат повертає `True`. Як тільки зустрівся перший елемент, для якого умова хибна, генератор негайно завершує роботу.
25. **Як працює dropwhile()?**
    `dropwhile(predicate, iterable)` пропускає всі початкові елементи, для яких предикат `True`. Як тільки умова вперше порушується, він повертає цей і всі наступні елементи до кінця послідовності без подальших перевірок предиката.
26. **Для чого accumulate()?**
    `itertools.accumulate` генерує потік накопичувальних значень (за замовчуванням наростаючу суму, або застосовуючи довільну бінарну функцію двох аргументів).

27. **Для чого pairwise()?**
    `itertools.pairwise` генерує послідовність сусідніх пар `(current, next)`. Це незамінно для обчислення дельт, різниць часових рядів та аналізу переходів між станами.

28. **Як працює groupby()?**
    `itertools.groupby(data, key)` групує **послідовні** сусідні елементи з однаковим значенням ключа `key`.

29. **Чому перед groupby() часто потрібне sorting?**
    Оскільки `itertools.groupby` не агрегує дані у глобальну хеш-таблицю (як SQL `GROUP BY`), а розриває групу, як тільки зустрічається елемент з іншим ключем. Якщо однакові ключі розкидані по датасету, виникнуть множинні розрізнені групи. Попереднє сортування гарантує, що всі однакові ключі розміщені підряд.

30. **Що робить product()?**
    `itertools.product(*iterables)` обчислює прямий декартовий добуток вхідних послідовностей (аналог вкладених циклів `for`).

31. **Чим combinations() відрізняється від permutations()?**
    `permutations` генерує перестановки, де порядок елементів має значення `(A, B) != (B, A)`. `combinations` генерує сполучення, де порядок не має значення і кожен піднабір унікальний `(A, B) == (B, A)`.

32. **Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?**
    Через факторіальну або експоненціальну комбінаторну складність. Навіть якщо генератор працює ліниво, кількість елементів може перевищувати трильйони, і повний обхід такого потоку триватиме роки процесорного часу.

33. **Що таке batching?**
    Це метод організації обробки потоку даних, за якого елементи накопичуються у невеликі пакунки (батчі) фіксованого розміру (наприклад, по 100 або 1000 шт.) для групової відправки в БД, мережу або на запис у файл.

34. **Які переваги batching?**
    Зменшення накладних витрат на мережеві виклики (network roundtrips), ефективніше використання системних викликів дискового вводу-виводу (I/O) та векторних операцій обчислення без надмірного навантаження на RAM.

35. **Що таке lazy pipeline?**
    Це ланцюжок з'єднаних між собою генераторів, де вихід попереднього генератора служить входом для наступного, а реальне обчислення відбувається за принципом «pull» під дією кінцевого споживача.

36. **Які stages може містити pipeline?**
    - Source (джерело читання);
    - Cleaning / Normalization (очищення);
    - Parser (синтаксичний розбір);
    - Validation (перевірка правил);
    - Filter (фільтрація);
    - Transformation (модифікація даних);

- Batching / Aggregation (пакетування та зведення);
- Sink (споживач результату: експорт, запис у БД, вивід на екран).

37. **Що є source у streaming pipeline?**
    Джерело (Source) — це початковий компонент, який генерує сирий потік даних (файл на диску, сокет мережі, черга повідомлень, сенсор або генератор).

38. **Що є transformation?**
    Трансформація — це проміжний етап конвеєра, який приймає елемент, модифікує його структуру або атрибути (наприклад, конвертує валюту, приводить рядки до регістру) та передає трансформований об'єкт далі.

39. **Що є sink?**
    Приймач (Sink) — це кінцева точка конвеєра, яка ініціює споживання ітератора («витягує» дані через виклики `next()`) і здійснює фінальну дію: записує результат на диск, виводить звіт або обчислює скалярні агрегати.

40. **Чи гарантує generator менший total execution time?**
    Ні, не гарантує. За рахунок накладних витрат інтерпретатора на перемикання контексту генератора повний обхід може бути дещо повільнішим за монолітний скомпільований код CPython для списків. Головна перевага генератора — радикальна економія RAM та низький TTFR.

41. **Яка основна memory advantage generator?**
    Константна просторова складність $O(1)$. Обсяг споживаної пам'яті визначається розміром одного поточного оброблюваного елемента і залишається фіксованим як для 10 елементів, так і для 100 мільярдів.

42. **Що таке time to first result?**
    Час до отримання першого результату (TTFR) — це тривалість від моменту старту запиту до моменту повернення клієнту першого готового обробленого елемента.

43. **Чому lazy pipeline добре підтримує early termination?**
    Тому що обробка кожного елемента відбувається строго по мірі потреби. Якщо умова пошуку виконана на першому ж елементі, конвеєр миттєво зупиняється через `return`, не витрачаючи ресурси на зчитування та парсинг решти мільйонів рядків.

44. **Чому exception у generator може виникати із затримкою?**
    Тому що код генератора фактично не виконується в момент створення об'єкта. Якщо синтаксично коректний генератор містить помилку обчислення (наприклад, `ZeroDivisionError` чи `ValueError`), вона проявиться лише тоді, коли споживач дійде до ітерації з цим значенням.

45. **Чому list(generator) може бути проблемою?**
    Виклик `list(generator)` змушує генератор миттєво віддати всі елементи, які розміщуються у списку. Це повністю знищує всі переваги лінивих обчислень і призводить до споживання сотень мегабайт пам'яті.

46. **Що трапиться при list(count())?**
    Програма зависне у нескінченному циклі додавання чисел у динамічний масив списку,

доки не вичерпає всю доступну оперативну пам'ять машини і не завершиться аварійно з фатальною помилкою `MemoryError`.

47. **Як generator може впливати на lifecycle file resource?**
    Якщо генератор відкриває файл у блоці `with` і робить `yield line`, файловий дескриптор залишається відкритим протягом усього часу існування ітерації. Якщо споживач кинув ітератор на середині і не викликав `.close()`, файл залишається заблокованим до моменту збирання сміття (GC).

48. **Як тестувати невеликий generator?**
    Перетворити його результат у список через `list(gen)` та перевірити відповідність очікуваній колекції через `self.assertEqual(list(gen), expected)`.

49. **Як тестувати infinite generator?**
    Використати `itertools.islice(gen, N)` для отримання фіксованої кількості перших елементів: `sample = list(islice(infinite_gen, 5))` і перевірити коректність згенерованого зрізу.

50. **Коли краще використовувати list, а коли generator?**
    - **Використовуйте `list`**, коли: набір даних невеликий і поміщається в пам'яті; потрібен багаторазовий швидкий доступ до елементів; потрібна довільна індексація за позицією `arr[i]`, зрізи списку, або визначення точної довжини `len(arr)`.
    - **Використовуйте `generator`**, коли: розмір даних великий або невідомий (файли, логи, мережеві потоки, результати SQL-запитів); критичним є пікове використання пам'яті; обробка побудована у вигляді послідовних стадій конвеєра; потрібна висока швидкість першої реакції системи (TTFR) або раннє переривання.

---

16. Загальний висновок

У ході виконання лабораторної роботи №3 було успішно досліджено та практично реалізовано парадигму лінивих обчислень та потокової обробки даних на основі ітераційної моделі Python:

1. **Реалізація архітектури потокового конвеєра:**
   Створено модульний, тестований конвеєр (Streaming Data Pipeline) для каталогу автомобілів (Варіант 5), що містить стадії потокового читання (`read_lines`), очищення (`clean_lines`), парсингу (`parse_csv_rows`), валідації з фіксацією помилок (`validate_cars`), нормалізації (`normalize_car_names`), фільтрації (`filter_by_brand_and_year_stream`), пакетної обробки (`batched_cars`) та однопрохідної аналітики (`streaming_statistics`).
2. **Опанування протоколу ітераторів та генераторів:**
   Розроблено власний клас ітератора `CarCatalogIterator` з iterable-контейнером `CarCatalogIterable`, рекурсивний генератор з `yield from` (`recursive_flatten`), безкінечний генератор даних на базі `itertools.count`, а також продемонстровано роботу інструментів `itertools.islice`, `chain`, `pairwise`, `groupby` та `accumulate`.
3. **Експериментальне підтвердження теоретичних оцінок:**
   Практичний бенчмаркінг на вибірках 10 000, 100 000 та 500 000 записів із застосуванням `tracemalloc` та `perf_counter` довів, що:

- Lazy Evaluation забезпечує стабільне, константне використання оперативної пам’яті $O(1)$ на рівні **~60.6 KiB**, демонструючи екологічність споживання пам’яті у **8 423 рази** порівняно з Eager (498.8*MiB*);
- Час реакції системи до першого результату (TTFR) у потоковому варіанті є практично нульовим (**0.2 мс** проти **1.96 с** на 500k рядків), забезпечуючи прискорення інтерактивності у **9 112 разів**;
- Завдяки ранньому перериванню (Early Termination) пошук цільового автомобіля завершився у **34.7 раза швидше** та заощадив у **1 159 разів більше пам’яті**.

4. **Якість коду та інженерна культура:**
   Збережено зворотну сумісність із попередніми лабораторними роботами, набір з 14 модульних тестів пройшов без помилок, звітність консолідовано в директорії `reports/`, додано виключення у `.gitignore`, а зміни оформлено атомарними комітами за стандартом Conventional Commits.